

Chapter 11 Trees

Chapter Summary

- Introduction to Trees
- Applications of Trees
- Tree Traversal
- Spanning Trees
- Minimum Spanning Trees

Section 11.1 Introduction to Trees

Section Summary

- Introduction to Trees
- Rooted Trees
- Trees as Models
- Properties of Trees

Trees

Definition:

A tree is a **connected undirected graph** with no simple circuit.

Definition:

A **forest** is a graph that has no simple circuit but is not connected. Each of the connected components in a forest is a tree.

Theorem:

An undirected graph is a tree if and only if there is **a unique simple** path between any two of its vertices.

Trees as Models

- Computer science, chemistry, geology, botany, psychology, and many other areas.
- Organization of computer file system

Rooted Trees

Definition:

A **rooted tree** is a tree in which one vertex has been designated as the **root** and **every edge is**

directed away from the root.

An unrooted tree is converted into different rooted trees when different vertices are chosen as the root.

Rooted Tree Terminology

- The **parent** of a non-rooted vertex v is the unique vertex u such that there is a directed edge from u to v . v is called a **child** of u . Vertices with the same parent are called **siblings**.
- The **ancestors** of a vertex are the vertex in the path from the root to this vertex. The **descendants** of a vertex are those vertices that have this vertex as a ancestor.
- A vertex without children is called a **leaf**. Vertex with children are called **internal vertices**.
- If a is a vertex in a tree, the **subtree** with a as a root is the subgraph of the tree consisting of a and its descendant and all edges incident to these descendants.

m -ary Rooted Trees

Definition:

A rooted tree is called an **m -ary tree** if every internal vertex has no more than m children. **Full m -ary tree** is the tree that every internal vertex has exactly m children. When $m = 2$, it is called **binary tree**.

Ordered Rooted Trees

Definition:

An **ordered rooted tree** is a tree where the children of each internal vertex are ordered. (show in order from left to right when drawing)

For binary tree, the first is called **left child** and the second is called **right child**. And corresponding subtree is called **left subtree** and **right subtree**.

Properties of Trees

Theorem 2:

A tree with n vertices has $n - 1$ edges.

Counting Vertices in Full m -Ary Trees

Theorem 3:

A full m -ary tree with i internal vertices has $n = mi + 1$ vertices.

Theorem 4:

A full m -ary tree

- n vertices has $i = (n - 1)/m$ internal vertices and $l = ((m - 1)n + 1)/m$

- i internal vertices has $n = mi + 1$ vertices and $l = (m - 1)i + 1$ leaves
- l leaves has $n = (ml - 1)/(m - 1)$ vertices and $i = (l - 1)/(m - 1)$ internal vertices.

Level of Vertices and Height of Trees

Definitions:

- The **level** of v in a rooted tree is the **length** of the unique path from the root to the vertex.
- The **height** of a rooted tree is the **maximum** of levels of the vertices.

Balanced m -ary Trees

Definition:

A rooted m -ary tree of height h is **balanced** if all leaves are at level h or $h - 1$.

The Bound for the Number of Leaves in an m -ary Tree

Theorem 5

There are at most m^h leaves in an m -ary tree of height h .

Corollary 1:

If an m -ary tree of height h has l leaves, then $h \geq \log_m l$. If it is full and balanced, then $h = \log_m l$.

Section 11.3 Tree Traversal

Section Summary

- Universal Address Systems
- Traversal Algorithms
- Infix, prefix and postfix notation

Tree Traversal

Procedures for systematically visiting every vertex of an ordered tree are called **traversals**. Most commonly: **preorder, inorder, postorder traversal**.

Preorder Traversal

T is an ordered rooted tree with root r . If T consists only of r , then r is the **preorder traversal** of T . Otherwise, $T_1, T_2, \dots, T_n$ are subtrees of r from left to right. The preorder traversal begins by **visiting r** , then **continues by traverse T_1 in preorder** until T_n .

Inorder Traversal

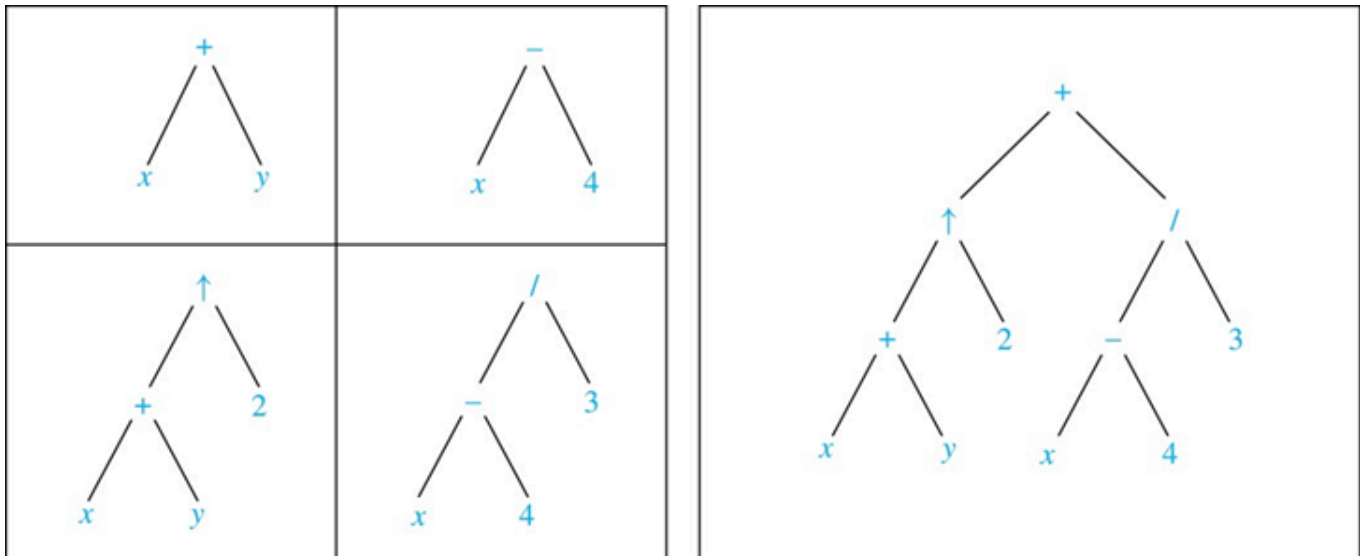
T is an ordered rooted tree with root r . If T consist only of r , then r is the **inorder traversal** of T . Otherwise, $T_1, \dots, T_n$ are subtrees of r from left to right. The inorder traversal **begins by traversing T_1 in inorder**, then **visiting r** and **continues by visiting T_2 in inorder** until T_n .

Postorder Traversal

T is an ordered rooted tree with root r . If T consist only of r , then r is the **inorder traversal** of T . Otherwise, $T_1, \dots, T_n$ are subtrees of r from left to right. The postorder traversal begins by traversing T_1 in post order, then T_2 in post order and continue until T_n , at last visit r .

Expression Trees

A binary tree for the expression can be built from the **bottom up** as is illustrated.



(expression $((x + y) * 2) + ((x - 4) / 3)$)

Infix Notation

Inorder traversal of the tree to obtain infix expression. (Parentheses are needed to ensure unique)

Prefix Notation

Preorder traversal of the tree to obtain prefix expression.

Operators precede their operands. Parentheses are not needed.

Calculate **from right to left**. When encountering an operator, we perform the corresponding operations with two operations **to the right**.

Postfix Notation

Postorder traversal of the tree to obtain postfix expression.

Operators follows its two operands. Parentheses are not needed. Calculate from left to right.

Section 11.4 Spanning Trees

Section Summary

- Spanning Trees
- Depth-First Search
- Breadth-First Search
- Backtracking Applications
- Depth-First Search in Directed graphs

Spanning Trees

Definition:

G is a simple graph. A **spanning tree** of G is a subgraph of G that is a tree containing **every vertex** of G .

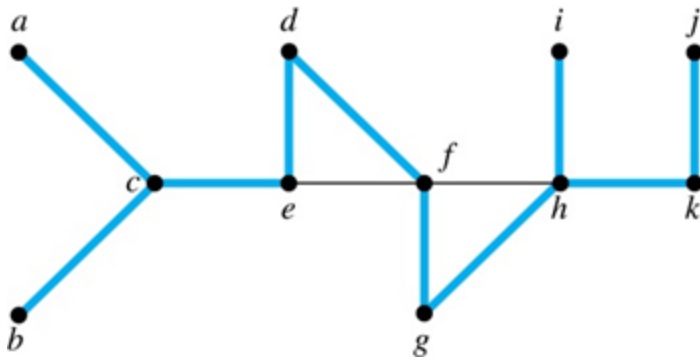
Theorem:

A simple graph is connected if and only if it has a spanning tree.

Depth-First Search

- Choose a vertex of the graph as the root
- Form a path starting at this vertex by **successively adding vertices and edges where each new edge is incident with the last vertex** in the path and **a vertex not in the path**. Continue adding as long as possible.
- If the pass goes through all vertices, the tree consisting of this path is a spanning tree.
- Otherwise, move back to the next to the last vertex in the path. If possible, form a new path starting at this vertex and passing through vertices not visited. If cannot done, move back another vertex in the path.
- Repeat until all vertices are included in the spanning tree.

The edges selected by DFS are called **tree edges**. All other edges are called **back edges**, which must connect a vertex to an ancestor or descendant of the vertex in the graph.



(The black edges are back edges)

Breadth-First Search

- Choose a root from the vertices of the graph
- Add all edges incident to this vertex and the other endpoint of each of these edges. These are at level 1.
- For each vertex added at previous level, add each edge incident to this vertex as long as it does not produce a simple circuit. The new vertices we find are at the next level.
- Repeat until all vertices have been added.

DFS in Directed Graphs

Both DFS and BFS can be modified to run on a directed graph. But the result may be a **spanning forest**.